Google Summer of Code 2026

Name: Nwakaego Glory Nwakaego

Project Title: Friendly Sketch Embedder project

Which method of communication do you prefer? (i.e. in person, email, chat, video conference, etc.): I prefer email, but I'm open to other forms of communication if needed.

What Processing Foundation GSoC project are you interested in? Friendly Sketch Embedder project

If you have a link to a resume/CV/LinkedIn profile, include it here.

Here's my portfolio, CV, Hashnode, and LinkedIn profile

Please describe yourself, including your development background and specific expertise.

I am someone who loves learning and growing. I like to describe myself as a lifelong student, always curious, always eager to take on challenges, and constantly looking for ways to improve.

I started out as a freelance writer, but over time, I found myself drawn to coding, especially frontend development. I have been learning JavaScript, HTML, CSS, and React.

Right now, I am focused on improving my skills in writing clean code, building user-friendly interfaces, and writing technical content that helps beginners and creators alike.

What do you hope to gain through the process of participating in GSoC, and specifically by contributing to the Friendly Sketch Embedder project?

Through GSoC, I hope to grow not just as a developer but as a problem solver and collaborator. I want to learn how to take an idea from research to reality while working with mentors and following community feedback.

Contributing to the Friendly Sketch Embedder project excites me because it's a chance to build something that helps others

Why are you interested in the Friendly Sketch Embedder project(s) you stated above?

The Friendly Sketch Embedder for p5.js really stood out to me because it solves a real problem. I can already see how working on this project will help me grow, not just as a developer, but also in research, communication, and technical writing.

I wanted to really understand what people struggle with, so I reached out to the p5.js community on Discord and LinkedIn. One of the best conversations I had was with Umar Khan, who shared this advice with me:

> "I focus on identifying and addressing the limitations of current p5.js embedding methods by researching user pain points, exploring hybrid solutions that balance simplicity and interactivity, engaging with the community for feedback, and iterating on a user-friendly, well-documented approach that improves performance, flexibility, and ease of use."

That really stayed with me and made me even more excited to build something helpful. I also spoke with Dave, who introduced me to the p5.js Widget. Using the widget taught me how important simplicity and accessibility are for teachers, artists, and students who just want things to work without too much setup.

But I also saw areas where things could be better, like giving users more control over their sketches and making it easier to customize and interact with them. That's what gave me the idea for my solution: I want to keep everything simple and beginner-friendly but add features that let users choose between iframe and script-based embeds, customize how their sketch looks, and add dynamic interactions like play, pause, and reset.

In the end, I hope what I build will help creators share their work more easily, and at the same time, help me become a better developer and problem-solver.

Have you reviewed the important dates and times for GSoC?

Yes, I have gone through all the dates and deadlines for GSoC and made sure they work with my schedule.

Do you have any significant conflicts with the listed schedule? If so, please list them here.

No, I don't have any conflicts. I have planned ahead so I can fully focus on this project.

Do you understand this is a serious commitment, equivalent to a full-time paid summer internship or summer job?

Yes, I completely understand. I am treating this with the same level of commitment and seriousness, and I'm ready to give it my best.

We strongly prefer students that provide code samples, ideally in a "social coding" site like GitHub. Please provide a link to code you've written, whether it's a zip file / tarball you host on your own or your GitHub profile.

Here's my GitHub profile. I am constantly updating it with projects as I learn and

build.

Overview

My main goal is to build an easy-to-use UI-based embed generator that outputs both iframe and script-based codes, a hybrid solution tailored for beginners who want to embed p5.js sketches with minimal technical setup.

While advanced customization options will be available, the interface and defaults will be designed primarily for beginner coders. To support this, script-based embedding will be prioritized, but iframe options will still be available for simple copy-paste functionality.

The project will be built with JavaScript (ES6+), HTML, and CSS. I will use React.js for creating a clean, dynamic UI due to my familiarity with it and its modular architecture. Lightweight interactions will be handled with vanilla CSS. Documentation will be clear, beginner-friendly, and written in Markdown.

Key User Goals and Prioritization

The focus is on making p5.js embedding simple and customizable without overwhelming

the user. All technical options (iframe vs script-based, instance vs global) will be presented only when relevant, and where possible, the best choice will be auto-selected based on user context (e.g., platform selection or use case).

Features will be implemented in a phased manner. The MVP (Minimum Viable Product) will focus on the most valuable use cases:

1. Easy generation of embed code (iframe first, script later).
2. Platform presets for simplified compatibility.
3. Custom embed UI (size, play mode, sound).

Advanced features (like JavaScript APIs and editable toggles) will follow once core embedding is in place.

Key Features and Improvements

Flexible Embed Code Generator

Users will choose between iframe and script-based embedding using a clear dropdown menu in the UI:

Embed Type → iframe / script

- The tool will default to "iframe" for ultra-simple, copy-paste scenarios.
- If "script" is selected, users will get cleaner performance and more control — this path is recommended for dynamic or multi-sketch setups.

To reduce cognitive load, only relevant options will appear based on the selected type:

If "iframe" is selected:

- Show width, height, background, border styling
- Enable platform presets (WordPress, Medium, etc.)

- Optionally use srcdoc for inline embedding

If "script" is selected:

- Show toggles like:
    - Use Instance Mode (recommended)
    - Expose Sketch API (play, pause, reset)
    - Make code editable
- Advanced technical options (e.g., global mode or code view) will remain hidden by default and only appear when “Show Advanced Options” is enabled.

Smart Platform Presets

Different platforms (WordPress, Medium, Notion, Ghost, etc.) have their own special rules for embedding, which can confuse beginners. To eliminate guesswork, the embed generator will include a “Platform” dropdown with smart defaults:

- Platform → WordPress / Medium / Ghost / Notion / Static Site / Other

When a platform is selected:

- The generator will auto-select the recommended embed type (iframe/script) and apply the safest settings.
- Users can still override choices manually by expanding “Advanced Options.”

Examples of Smart Defaults:

- WordPress / Medium → iframe with srcdoc (to bypass script-blocking)
- Notion / Ghost → hosted iframe (HTTPS required)

- Static Sites / Developer blogs → full script-based embedding (for maximum flexibility)

This reduces trial-and-error and supports a clean “choose platform → copy embed” workflow for beginners, while still giving advanced users full control when needed.

Dynamic Sketch Controls & Styling (Simple UI + Advanced Power)

To give users full creative control, the embed generator will provide two powerful sets of options one for how the sketch behaves and one for how it looks.

Dynamic Behavior Controls

These let users decide what their sketch *does*:

- Play / Pause / Reset buttons will appear on the embedded sketch *if the user wants them*.
- These buttons are helpful for interactive or looping sketches — especially for artists and educators who want users to explore at their own pace.

Beginners can toggle these controls on or off using simple checkboxes.

For developers or advanced users, a JavaScript API will also be available, but only shown if "Advanced Options" is enabled:

```
function pauseSketch() { running = false; }

function playSketch() { running = true; }

function resetSketch() { myP5.resetSketch(); }
```

These functions give more control over sketch behavior in Instance Mode.

Custom Styling Options

While behavior controls how the sketch *acts*, styling controls how the sketch *looks*. Users can customize the visual appearance using an intuitive, no-code UI:

- Resize the sketch (width and height sliders)

- Add rounded corners, drop shadows, and padding
- Set background colors (with a color picker)
- Switch between light/dark themes to match their website

All styling updates will reflect live in a preview window, so users can immediately see what the embedded sketch will look like.
Together, these two panels, Behavior and Styling, ensure that users can both fine-tune how their sketch performs *and* make it look great, all without needing to write code.

Automatic Library Detection

When embedding sketches outside the p5.js Web Editor (e.g., on WordPress, Notion, or GitHub Pages), users often forget to include extra libraries like p5.sound.js. This tool will handle that for them.

If a user selects a sketch type that uses sound, for example, the generator will automatically add the correct <script> tag for p5.sound.js in the embed code. This removes confusion for beginners and ensures their sketches run correctly across platforms.

Asset Management:
Handling external files (like images, sounds, or fonts) is one of the biggest challenges when embedding p5.js sketches on other platforms. To make this easier:

- I will recommend beginner-friendly hosting platforms such as GitHub Pages or the p5.js Web Editor for uploading assets.

- I will provide simple tutorials and examples on how to link those assets in a sketch.

- When possible, the tool will offer an option to generate srcdoc-based iframes that include the entire sketch and base64-encoded small files. This will let users copy one embed code that works anywhere, no extra uploads required.

These steps will ensure a smoother experience for both beginners and advanced users who want full control of their sketch environment.

Clean and Beginner-Friendly Code Output

To avoid overwhelming beginners, the embed tool will keep things simple by default, but still powerful for advanced users.

Beginner Flow (Default)

- The user only sees a clean, working embed code ready to copy and paste.
- The code is minified (short and tidy) so it doesn't look intimidating.
- If the user is embedding more than one sketch, the tool will automatically use Instance Mode under the hood to prevent sketch conflicts, no extra setup required.
- Users don't need to worry about technical details like script tags or namespace collisions.

Advanced Flow (Optional)

When the user clicks "Advanced Mode," more customization options appear: ● A

"Show Full Code" toggle reveals the complete script-based embed code.

- The user can choose between:
    - Global Mode (uses global functions like setup() and draw()).
    - Instance Mode (uses scoped variables to avoid name clashes).
- The tool will explain which mode is better depending on the use case (e.g., multiple sketches = Instance Mode).

Example Embed Codes: Global Mode:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/p5.min.js"></script>
<script>
```

```
  function setup() {
    createCanvas(600, 400);
    background(220);
  }
  function draw() {
    fill(255, 0, 0);
    ellipse(mouseX, mouseY, 50, 50);
  }
</script>
```

Instance Mode:

```
<div id="mySketch"></div>
<script src="https://cdnjs.cloudflare.com/ajax/libs/p5.js/1.9.0/p5.min.js"></script>
<script>
  let sketch = function(p) {
    p.setup = function() {
      p.createCanvas(600, 400);
      p.background(220);
    };
    p.draw = function() {
      p.fill(255, 0, 0);
      p.ellipse(p.mouseX, p.mouseY, 50, 50);
    };
  };
  let myP5 = new p5(sketch, "mySketch");
</script>
```

No-Code UI for Embed Customization. The interface will allow users

to: ● Choose embed type (iframe or script)

● Adjust sketch size

- Toggle play/pause/autoplay behavior
- Enable live preview
- Show/hide advanced code view
- Toggle code editability in the embed

Smart Platform Presets (Platform-Aware Embeds)

Different platforms have different limitations. For example, WordPress and Medium block custom JavaScript, while static sites allow more flexibility. This tool will include a dropdown of popular platforms (WordPress, Medium, Notion, Ghost, etc.), and adjust the embed code accordingly.

- WordPress/Medium: Will use an iframe with a srcdoc attribute to include the full sketch inline.
- Static Sites (e.g., GitHub Pages, Netlify): Will output a simple embed using <script> tags, either in Global or Instance mode.

A “Copy Code” button with visual feedback will reduce friction, especially for beginners.

Documentation Plan Clear, beginner-friendly documentation will be a priority. It will include:

- Screenshots
- Step-by-step setup for each platform
- Hosting assets on GitHub or p5 editor
- Troubleshooting common issues (e.g., iframe blocked)

Timeline Adjustment: Emphasizing Design Phase Week 1:

- Conduct user research (target audience: beginner coders, teachers, artists)
- Define core use cases
- Create wireframes and testable UI mockups
- Gather community feedback before implementation

Why This Project Matters This tool bridges the gap between beginner-friendly sketch sharing and customizable, interactive embeds. By focusing on ease of use and platform compatibility while giving power users deeper control, it brings p5.js closer to its community's diverse needs.

Deliverables Summary

- Hybrid embed generator (iframe + script)
- Beginner-first UI with live preview
- Smart presets per platform
- Script API for play/pause/reset
- Design phase with mockups + feedback
- Accessibility and performance improvements
- Clear documentation with tutorials

Mentor Feedback Integration

This proposal directly incorporates feedback, to ensure the scope, audience, and implementation are realistic and beginner-friendly. Here's how that feedback shaped the proposal:

Mentor Feedback	How It's Addressed
Focus on beginner users who just want to embed quickly	Nice-to-Have, and Stretch. The prototype will focus only on Core features, and others will be prioritized based on user feedback and timeline.
Don't overwhelm users with too many options	The proposal discusses using platforms like GitHub, Glitch, or p5 Web Editor to keep things backend-free and user-controlled. Asset management for iframes is flagged as a challenge and will be handled with clear documentation and beginner-friendly solutions.
Think through asset hosting and management	The tool will automatically minify and wrap output code so beginners can copy a clean version. Full code will be available separately for advanced users.
Minify complex code like Instance Mode before displaying	Week 1 of the timeline is dedicated to designing the tool: user personas, mockups, and interface flow testing. No code will be written before this phase is complete.
Include a design phase before development begins	
The tool defaults to script-based embedding with auto-selected settings to reduce decision fatigue. Technical differences like iframe vs script are hidden unless advanced users want to explore them.	
Features are broken down into Core,	

My Approach to Documentation

Here’s how I will make sure this tool is not only built, but easy for everyone to use.

One thing I have come to realize while planning this project is that building a tool is just

one part of the job, the other equally important part is making sure people can actually understand and use it with ease. That's why I plan to put real effort into writing documentation that's clear, simple, and beginner-friendly.

I want to create step-by-step guides that walk users through everything, from choosing canvas settings like width, height, and background color, to picking between iframe and script-based embedding.
I will also explain how to copy and paste those generated codes into platforms like WordPress, Medium, or static site builders, because I know not everyone will be comfortable figuring that out on their own.

I am also going to add a troubleshooting section to help answer common questions or fix small mistakes that might pop up.

All of this documentation will live on a dedicated GitHub Pages site, or maybe even on the p5.js website if that's possible. My goal is to make it easy to find and simple to follow.

Pre-Midterm (Weeks 1–6)

Prework

• Research existing embed solutions and common use cases.

• Engage with the p5.js community to identify real user needs.

• Set up development environment and finalize project scope with mentor input.

Phase 1: Design & Initial Setup (Weeks 1–2)

• Design and prototype the UI (e.g., mockups, wireframes).

• Review designs with mentor and get feedback from the

community. • Finalize user flow for generating embed code.

• Begin development of the core embed generator using React.

• Introduce beginner-friendly defaults (e.g., show iframe by default; script mode hidden under “Advanced”).

• Gather early feedback through test sketches.

Phase 2: Iframe Embedding Features (Weeks 3–4)

• Implement iframe generation with responsive height resizing (using postMessage).
• Add accessibility features like title attributes.

• Ensure compatibility with popular platforms (e.g., WordPress,

Notion). • Begin writing internal documentation and guides.

Phase 3: Script-Based Embedding & Static Site Testing (Weeks 5–6) •

Implement support for both Instance and Global mode sketch embedding.

• Add optional API controls (pause, play, reset).

• Refine the UI for advanced settings toggle.

• Test implementation with static site builders (e.g., GitHub Pages, Netlify).

• Research asset management strategies for users embedding images, fonts, and sounds (e.g., external hosting vs. bundling).

• Document best practices for asset linking and sketch hosting.

Post-Midterm (Weeks 7–12)

Phase 4: Platform Presets & Live Previews (Weeks 7–8)

• Add preset configurations for common platforms (e.g., Medium, WordPress,

Notion). • Build a live preview panel so users can see how their embed will appear. •

Continue gathering feedback and refining UI/UX.

Phase 5: Documentation & Tutorials (Weeks 9–10)

• Write complete user guides, how-tos, and FAQs tailored to different user

levels. • Conduct extensive cross-platform testing.

• Fix bugs and edge-case issues found through user feedback or

testing. Phase 6: Final Testing & Cleanup (Week 11)

• Perform final polish and UX refinements.

• Add automatic minification to generated script-based code (to hide boilerplate from beginners).

• Publish documentation and create a project blog post on the development journey and impact.

Wrap-Up (Week 12)

• Address final mentor/community feedback.

• Submit final evaluation and ensure all deliverables are complete.

• Ensure long-term usability and ease of maintenance for the tool.

Post-Project Engagement

• Remain active in the p5.js repository to support users.

• Help maintain and improve the embed generator over time.

• Continue learning and contributing to the creative coding community.

My Availability

As a freelance writer and developer, I have full flexibility to dedicate the necessary time to this project. I will allocate 10–12 hours per week, ensuring steady progress. If needed, I will adjust my schedule to accommodate additional work.

What I Hope to Gain from My Mentor

I seek guidance in:

- Technical Best Practices: Code reviews, performance improvements, and scalable architecture.
- User-Centric Design: Ensuring the solution truly addresses user needs. ● Project Management & Community Engagement: Learning how to maintain open-source contributions effectively.
- Regular Feedback & Collaboration: To refine the tool continuously.

Why Me

I am passionate about building tools that simplify creative coding. As a self-taught developer, I have learned JavaScript, React, and p5.js, allowing me to create interactive projects. My experience as a freelance technical writer also helps me explain complex ideas clearly.

This project is shaped by real community feedback, and I am committed to making embedding p5.js sketches more accessible. I look forward to working with mentors and contributing meaningfully to the p5.js ecosystem.